`BountyPulse`: Decentralized Micro-Bounty & Escrow

Project Details

- **Submission & Presentation Date: August 11, 2026**
  (Individual Viva & Group Presentation).
- Submission form: Link (**Form closes: Aug 11, 11 AM**)

- **Evaluation & Marking Distribution**
  (Total Weight: 12.5 Marks / 50% of Total Lab Grade):

    - Solidity Smart Contract Implementation: **1.5 Marks**
    - Smart Contract and Solidity Viva: **3.0 Marks**
    - DApp Frontend & IPFS Integration: **2.5 Marks**
    - ETH DApp and IPFS Viva: **5.5 Marks**

1. System Overview & Context

In the traditional freelance and gig economy, centralized platforms take massive commission fees (up to 20%), lock user accounts arbitrarily, and own the database of user reputations and work history.

You are tasked with building **`BountyPulse`**, a decentralized Web3 micro-bounty platform.

- **The Storage Layer:** To prevent **Blockchain State Bloat**, heavy data (profile avatars, project descriptions, and final submitted work files) must be pinned off-chain to the **InterPlanetary File System (IPFS)** via the **Pinata API**, storing only the resulting 46-character Cryptographic Content Identifiers (**CIDs**) on the blockchain.
- **The Settlement Layer:** An EVM smart contract manages the registry of users, handles escrowed funds securely, calculates platform fees, and enforces reputation penalties.

2. Part A: Smart Contract Specifications

Your backend logic must be written in **Solidity `^0.8.20`** and compiled/deployed using **Foundry (Forge)** on a local **Anvil** node. Y**ou must design your own Enums, Structs, and Mappings based on the following requirements.**

2.1 System Roles & Registry

1. **`Arbiter` (Admin):** The platform overseer (contract deployer) who resolves disputes and collects protocol operational fees.

2. **`Client`:** Users who post bounties (gigs) and lock ETH into escrow.
3. **`Freelancer`:** Users who bid on bounties, submit work, and earn ETH.

Registration Requirements:

- All users must register their `Name`, `Role`, and an `ipfsAvatarHash`.
- The smart contract must act as the sole database (Registry).
  A wallet address cannot be registered twice.
- `Freelancer` accounts must automatically be initialized with a **Reputation Score of 100**.

2.2 Core Application Logic

1. Post a Bounty (Client)

- `Client` accounts can post bounties requiring a `Max Budget(in Wei)` and an `ipfsBountyDetailsHash` (containing the gig description).
- The bounty's initial status is `Open`.

2. The Bidder Registry (Freelancer Quotes)

- `Freelancer` accounts submit a bid (a price quote in Wei) to complete an `Open` bounty. **This function must be non-payable;** freelancers do not send ETH to bid; they only propose their asking price for the job.
- **Constraint (Budget Ceiling):** The freelancer's proposed asking price cannot exceed the `Max Budget` set by the client.
- **Constraint (Reputation Gate):** A freelancer cannot submit a bid if their Reputation Score is **below 40**.

3. Escrow Funding & Revert Logic (`payable`)

- The `Client` selects a winning bid and triggers the funding function, which requires them to send ETH.
- **Strict Payment Constraint:** The Client must send the exact ETH amount of the chosen bid.
  - If they send *less* than the bid amount, the transaction must revert entirely.
  - If they send *more* than the bid amount, the contract must accept the exact bid into escrow and **refund the excess ETH** back to the Client in the same transaction.
- Status updates to `Locked`.

4. Work Submission & Percentage Math

- The `Freelancer` submits an `ipfsWorkFileHash` to the contract.

- The `Client` approves the work.
- **Operational Cost Calculation:** The contract must deduct a **2% platform fee** from the escrowed amount and allocate it to the `Arbiter`.
- **Fund Tracker (Pull-Payment Pattern):** Do NOT automatically send the remaining 98% ETH to the freelancer. Instead, add it to the freelancer's `Withdrawable Balance` mapping.
- Reputation increases by **+15 points**. Status updates to `Resolved`.

5. Claiming Funds

- The `Freelancer` or `Arbiter` must call a specific `claimFunds` function to withdraw their accumulated balances to their actual MetaMask wallet.

6. Dispute, Refund & Penalty

- If the work is unsatisfactory, the `Client` marks it as `Disputed`.
- The **`Arbiter` resolves it**:
    - **Outcome A (Freelancer Fault):** 100% of the escrow is refunded to the `Client`. The freelancer suffers a **-30 point reputation penalty**.
    - **Outcome B (Client Fault):** The promised amount is added to the Freelancer's Withdrawable Balance (minus the 2% fee).

3. Part B: Decentralized Application (DApp) & IPFS Interface

Build the client-side UI using **HTML/CSS/Vanilla JavaScript** (or React/Vue), communicating via **Ethers.js v6**.

3.1 Role-Based UI & Auto-Wallet Detection

- The UI must automatically detect the active MetaMask address on load (*do not use text inputs for wallet addresses*).
- Based on the on-chain Registry, dynamically display the correct dashboard (Client, Freelancer, or Arbiter).
- Implement `window.ethereum.on('accountsChanged')` so the UI changes instantly if the user switches accounts in MetaMask.

3.2 View Operations & Sorting

- Fetch and display the Registry of open bounties and submitted bids dynamically from the blockchain.

- **Sorting Constraint:** You must implement a filter/sort feature (e.g., "Sort by Highest Budget"). *You must demonstrate the gas-optimization principle.*

3.3 Two-Step IPFS Pipeline (`ipfsHelper.js`)

- Forms (Registration, Posting Bounties, Submitting Work) must capture files, upload them to Pinata via HTTP POST, receive the CID, and pass that CID to the smart contract.
- Images and text must render dynamically in the UI using an IPFS Gateway `https://gateway.pinata.cloud/ipfs/<CID>`.

3.4 Interactive Fund Tracking

- Freelancers and Arbiters should see a specific "Unclaimed Earnings" tracker on their dashboard.
- Include a **"Claim Funds"** button that triggers a MetaMask transaction to withdraw their available balance.

3.5 Real-Time Event Syncing

- The UI must update reactively using Ethers.js event listeners (`contract.on(...)`).
- If a status changes or a bid is placed, the UI must update without requiring a hard page reload (`window.location.reload()`).

4. Sequential Implementation & GitHub Collaboration Guide

Since your midterms are currently ongoing, it is highly recommended that you distribute this workload and tackle it sequentially in your free time.
Think about the end goal: **a working Project by August 11th.**

You must use **GitHub** to manage your codebase. Below is a sequential guide for how your group should conquer the project, task by task, so you can work asynchronously during your exam gaps.

> **Important Note on Marking:**
> The implementation code is only worth 4.0 marks. The remaining 8.5 marks belong to the Viva. If you rely on AI to write your logic without understanding the underlying logic, state changes, and Web3 architecture, you will fail the Viva.

Recommended Collaboration Flow:

- **Member 1 (Smart Contract Engineer):** Focus on Checkpoints 1 & 2. Design the structs, handle the percentage math, and enforce the exact-payment constraints. Push the ABI to GitHub.
- **Member 2 (Frontend & Storage Dev):** Focus on Checkpoint 3. Build the UI forms, handle the auto-wallet detection, and build the Pinata IPFS uploads.
- **Member 3 (Web3 Integrator):** Focus on Checkpoint 4. Use Ethers.js to pull view data from the contract, execute the sorting logic, and connect the "Claim Funds" and "Pay Escrow" buttons to MetaMask.
- **Member 4 (UX/UI Polish):** Focus on Checkpoint 5. Implement the Ethers.js event listeners so the app feels like a modern, real-time application without page reloads.

Implementation Checkpoints & Task Checklist:

- **Checkpoint 1 (Environment) — *[0.0 Marks - Prerequisite]***
    - Anvil running locally, MetaMask connected to Chain ID 31337, pre-funded test accounts imported, and Pinata JWT active.
- **Checkpoint 2 (Contract Deployment) — *[1.5 Marks]***
    - Successful deployment of `BountyPulse.sol` using Forge, showing terminal output, gas metrics, and contract address. Proof of correct percentage math and revert logic.
- **Checkpoint 3 (IPFS Metadata Pipeline) — *[1.0 Mark]***
    - Code walkthrough and live demonstration of the frontend successfully pinning an image or text file, receiving the CID, and passing it to the blockchain.
- **Checkpoint 4 (Feed & Escrow Flow) — *[1.0 Mark]***
    - Successfully fetching, sorting, and rendering data from the contract. Executing an exact-match ETH escrow payment and successfully utilizing the "Claim Funds" pull-payment button.
- **Checkpoint 5 (Live Event Auto-Sync) — *[0.5 Marks]***
    - Side-by-side browser test. A Client approving work in Window 1 instantly updates the Freelancer's "Unclaimed Earnings" balance in Window 2 without reloading the page.